osmo-operator

documentation, configs, résultats et logs — /opt/GSM/osmo-operator

Banc GSM émulé — banc-max

2026-10-28

Table des matières

1 Synthèse

run	élément	valeur
(pas de verdict.txt)		

dossier de run	contenu
Verdict et couverture	échelle des barreaux et tableau de couverture couche 1
Résultats des tests	captures VTY, diagnostics, rapports pytest, grafcets (.txt .md .mmd)
Fichiers	sources, scripts, configs et docs du banc (tout fichier texte)
Logs	journaux (.log), en dernier

i Méthode

Dossiers : /opt/GSM/osmo-operator. Extensions : tous les fichiers texte (hors .git, caches, binaires, sauvegardes, LICENSE et sorties précédentes). Entrée 1134 Ko, sortie 1084 Ko.
Compactage sans perte : fichiers identiques cités une fois ; lignes consécutives identiques « ×N » ; lignes ne différant que par des nombres : gabarit avec ⟨k⟩ puis la liste ordonnée des valeurs (k-uplets), chaque ligne se reconstruit en remettant les valeurs dans l'ordre. Seuls les codes couleur, la négociation telnet des captures VTY et les caractères de commande sont retirés ; les lignes de plus de 1000 caractères sont coupées en morceaux terminés par ↩. Les .md sont inclus tels quels (titres rétrogradés) et les .mmd rendus en diagrammes Mermaid, sans compactage.

2 Resultats des tests

2.1 /opt/GSM/osmo-operator/README.md

14376 octets, 177 lignes

2.1.1 osmo-operator

A complete **GSM + LTE operator on one machine**: an Osmocom core network (BTS, BSC, MSC, HLR, MGW, SMSC, Asterisk) and **emulated Calypso phones**. The ARM runs under QEMU (qosmo) and the TI C54x DSP runs outside it, executing the real mask ROM (c54x_exe). The phone is either the OsmocomBB `layer1` + `mobile`, or a Motorola C139 running the **original TI firmware** (FreeCalypso Magnetite). It camps, authenticates, ciphers, sends SMS and holds voice calls. No SDR and no physical phone: the radio is emulated end to end, down to the baseband.



Figure 1: The bench in action: CSFB call, I/Q spectra, srsUE, Wireshark, Linphone

2.1.1.1 Start here

I want to…	Read
Boot a ready-made live image (fastest, nothing to build)	iso · latest release
Pick a mode and run it	Modes below · the **?** of the dock (`osmo-info`) · lancer-les-modes.ipynb
Install on Ubuntu 24.04 without Docker	native · installer-osmo-jusquau-dsp.ipynb
Run in Docker, or build the images	docker
Run on a Raspberry Pi 4 / arm64	arm
Study the real C54x DSP on the mask ROM	dsp
Run the original TI firmware (C139, Magnetite)	magnetite
Use the gr-gsm layer 1	grgsm
Add 4G, CSFB and the postmarketOS phone	lte
Several operators, SS7 interconnect, a WAN between machines	ss7 · WAN
Drive the bench day to day: tmux, VTY, ports, Wireshark	usage
Check that the bench is healthy, reproduce a fault	tests
See how the pieces fit	architecture · all documents

2.1.1.2 Modes

Everything goes through `./start.sh` (`start-native.sh` is a symlink to it). `--regen` rebuilds `/etc/osmocom` from the templates, `--no-attach` stays out of tmux, `./start.sh --docker` hands over to `launch/start-docker.sh`, `./start.sh --help` lists every option.

Command	What runs	State
`./start.sh --bts-classique --ms`	**Default** (red phone icon). `osmo-bts-trx` ↔ `pont/pont_dsp.py`; the phone runs osmocom-bb `layer1` + `mobile` on the DSP ROM	LU, SMS, calls
`./start.sh --bts-classique --ms-magnetite`	Same network, the phone is a C139 running the TI firmware; `--gapk` for voice on the host	LU, SMS; calls in A5/0 (`ENCRYPTION="a5 0" ./start.sh --regen …`)
`./start.sh --bts --ms --ts 2`	The Calypso of QEMU **is the BTS radio** (`trx` firmware + `transceiver`); a second Calypso camps on it over the air; one Calypso per timeslot	camps, LU; no call (see Known issues)
`./start.sh --bts --ms --ts 2/1`	TS0 + TS1 on one Calypso TRX (default of `--bts`)	calls
`./start.sh --grgsm`	gr-gsm layer 1 instead of the DSP	end to end
`banc-rssi`	The osmocom-bb `rssi` app on a 4th Calypso, fed by the bridge; screen `c123-3`	DL levels, sync, BCCH

Command	What runs	State
`./start.sh --stop`	Stop the whole stack	

Phone screens and keypads (`scripts/osmo-banc-aliases.sh`): `c123` = instance 1 (phone of `--bts-classique`, or TRX of `--bts`), `c123-1` = instance 2 (phone over the air), `c123-2` = instance 3 (second TRX), `c123-3` = instance 4 (rssi), `c123-mobile` = the first live instance that is not a TRX, `c123-tous` = all of them. The underlying command is `tools/osmo-c123-gtk.py --instance N`. Nothing telecom starts at boot (`services/80-osmo-operator.preset`): starting a network is your gesture.

Quickest path from the ISO (password `osmo`):

```
cat osmo-operator-desktop.iso.part-* > osmo-operator-desktop.iso
sha256sum -c --ignore-missing SHA256SUMS
qemu-system-x86_64 -cdrom osmo-operator-desktop.iso -m 8G -enable-kvm -cpu host -smp 4
```

From a clone (native, or inside the container):

```
./start.sh --regen --bts-classique --ms   # first run: generate the configs, start, attach to tmux
./start.sh --stop
```

2.1.1.3 Notebooks

Notebook	What it does
installer-osmo-jusquau-dsp.ipynb (nbviewer)	Fresh Ubuntu 24.04 to a call on the DSP, one cell per step with checks; optional Magnetite build
lancer-les-modes.ipynb (nbviewer)	Each mode above: start, check, stop; the phone screens inline, keys, `banc-rssi`
lancer-pas-a-pas.ipynb (nbviewer)	The bench process by process: HLR, MGW, MSC, BSC, BTS, QEMU, osmocon, mobile, bridge

How to run them (disposable container, VM, unattended `nbconvert`): README-notebook.

2.1.1.4 Supplements

What is not in the ISO installs with `sudo ./addition.sh` (a checklist window, or flags):

Flag	Installs
`--multi`	Docker, the operator image, the SS7 multi-operator topology
`--opencl` · `--claude` · `--extras` · `--sim`	GPU OpenCL and A5 tools · Claude Code · games and media · PC/SC SIM tools
`--gnuarm`	the historic OsmocomBB toolchain, arm-elf-gcc 4.5.2 (`/root/gnuarm`, added to `PATH` by `/etc/profile.d/97-osmo-gnuarm.sh`)
`--magnetite`	Wine + TI TMS470, FreeCalypso Magnetite for the C139, the bench patches, the flash image
`--trx`	osmocom-bb-trx: the Docker image and the `trx` firmware (TX) built with gnuarm
`--calypso-fw`	the bench's Calypso firmware (`layer1`, `rssi`) rebuilt with gnuarm (prebuilt in firmware)
`--fernvale`	MediaTek MT6260: fernly, fernvale-nuttx, osmocom-bb-mtk-layer1
`--simcom`	the SIMCom documentation and SDKs
`--firmwares`	all the firmware supplements above
`--all`	Docker, image, multi, OpenCL, Claude, extras, SIM
`--status`	show what is installed, install nothing

2.1.1.5 Repository layout

Path	Purpose
`start.sh`, `start-native.sh` (symlink)	the launcher: loads the environment, picks the mode, generates `mobile_*.cfg`, then runs `/opt/GSM/qosmo/run.sh`; `--docker` hands over to `launch/start-docker.sh`
`startup_scripts/`	the launcher's steps, sourced in order by `start.sh` (options, node identity, environment, DSP bench menu, mobile plan, config regeneration, start)
`launch/`	the other launchers: `start-docker.sh` (containers, N operators), `start-interstp.sh` (the SS7 hub node), `start-multi.sh` (native + Docker operators), `start-nitb.sh`, `start-oqc.sh`, `osmo-launch.sh`
`environment/`	`load.env`, `paths.env`, `modes.env`: variable precedence and path resolution (README)
`globals.conf`, `globals.conf.example`, `generate_configs.sh`	the network settings (ARFCN, PLMN, …) and the only script that substitutes them into the templates
`configs/`	configuration templates for every daemon (`osmo-*.cfg`, Asterisk, open5gs, `gprs/`, `conky/`), wallpapers and screenshots
`lib/`	`gabarits.sh` (template substitution), `audio.sh` (PulseAudio + GAPK chain)
`pont/`	the radio bridges between the emulated Calypso and `osmo-bts-trx`: `pont.py` (gr-gsm), `pont_dsp.py` (DSP), `air.py`/`airmesh.py` (over-the-air between Calypsos), `dsp/` (README)
`opt-gsm/`	gr-gsm side helpers: `si_bridge.py`, `grgsm_fft_live.py`, `record_drain.py`
`scripts/`	run-time services and diagnostics: SMS relay (`sms-interop-relay.py`), audio (`gapk-start.sh`, `audio-diag.sh`), `call-diag.sh`, `banc-repro.sh`, `egprs-dsp.sh`, `magnetite-c139.sh`, the bench aliases, the Docker entrypoint and in-container `run.sh`
`checks/`	read-only health checks of a running stack (`check_all.sh`, `ss7_check.sh`, `diag-*.sh`, `annuaire.sh`)
`network/`	WAN table, mesh, firewall, VirtualBox interconnect, ISM payload, IP plan, loopbacks
`navigation/`, `ss7-console.py`	the SS7 console (MAP/TCAP/SCCP/M3UA, TUI, diagnostics); `ss7-console.py` at the root is a shortcut (QUICKSTART)
`helpers/`, `set_stp_ip.sh`	inter-STP config generator, host preparation, SS7 addressing of a VM
`tools/`	user tools: phone screens (`osmo-c123-gtk.py`), FFT viewers, `osmo-info`, `osmo-launcher.py`, settings page (`osmo-reglages.py`), LTE/pmOS/SIM installers, `osmo-patches-apply.sh`, `osmo-firmwares-install.sh`, documentation generators (`full_qmd.sh`, `full_text.sh`, `dockerfile-to-install-md.sh`)
`patches/`	patches applied on top of the upstream trees under `/opt/GSM` by `tools/osmo-patches-apply.sh` (idempotent; used by `install_modules/40-patches.sh`, `iso_modules/70-scripts.sh`, `update.sh` and `addition.sh --calypso-fw`)
`install.sh`, `install_modules/`	native installation by steps (`prereqs`, `deps`, `user`, `sources`, `patches`, `build`, `binaires`, `configs`, `bureau`, `verify`)
`addition.sh`, `tools/osmo-*-install.sh`	the supplements above
`update.sh`	ISO session start: login animation and patch catch-up on an installed machine

Path	Purpose
`Dockerfile`, `Dockerfile.lite`, `Dockerfile.run`, `Dockerfile.stp`, `build.sh`, `compose.sh`, `compose.yaml`	the Docker images and Compose topology (README-docker)
`packaging/`	the `.deb` cache (`osmo-deb.sh`), standalone packages (`build-debs.sh`), apt mirror helpers
`build-iso.sh`, `iso_modules/`, `installer/`	the live ISO, the SD image and the Calamares installer (README-iso, wiki/ISO)
`armbian/`	the same arm64 image through the official armbian/build framework (README-arm)
`services/`	shipped systemd units and the boot preset (README)
`data/`	desktop entries, icons, phone photos and click zones (`c123*`, `c139*`), node/operator name tables
`instances/`	per-Calypso-instance settings (`instance-N.env`, written by the settings page)
`web/`, `fft-web/`	the settings web page and the two-spectrum FFT web server (port 8081)
`doc/`	topic READMEs, `magnetite.md`, step-by-step install guides
`wiki/`	the wiki pages and the notebooks
`tout-en-un.py`	gathers the text files of a tree into one shortened Markdown report

2.1.1.6 Wiki

Page	Contents
Home	quick start, launching, reference card
Start-native	the launcher step by step, option manual
Reference	architecture, layer-1 chains, build graph, multi-PLMN SS7, diagnostics
Build · ISO	`build.sh`, `build-iso.sh`, `.deb`, `install.sh`; what the ISO contains
Environnement	which variables win, what reaches QEMU
Resultats	reference `check_all.sh` runs and how to read the deviations
Telephone-emule	plug **only** the emulated phone into your own Osmocom stack
Lancer-a-la-main	the phone started by hand, with proof at each step
Installer-qosmo · Installer-c54x_exe · Installer-grgsm_exe	building the QEMU fork and the two external layer 1s
SIM	a sysmoISIM in an Omnikey reader, or a softSIM, over SAP
WAN · Payload-ISM	linking nodes; SMS text over an ISM radio link

2.1.1.7 Other documents

Document	Subject
`pont/README.md`	the radio bridge: A5 ciphering, Kc and SACCH
`doc/magnetite.md`	the TI firmware in qosmo: build, flash image, running, debugging
`environment/README.md`	environment variables and path resolution
`services/README.md`	the systemd units shipped, and the boot policy
`navigation/QUICKSTART.md`	the SS7 console
Components and their READMEs	every Osmocom module and third-party component, with the local patches